

FACTORIZATION MACHINES

AP

FACTORIZATION MACHINES

GENESIS

Invented by [Steffen Rendle](#), now Google Research:

- [2010 IEEE Int'l Conf. on Data Mining](#)
- [ACM Trans. on Intelligent Systems and Technology \(TIST\) 3 \(3\), 57](#)

PROBLEM STATEMENT

Instance:

- a collection (dataset) $\mathbf{D}$ of m numerical datapoints (points in $\mathbb{R}^n$)
- a classification system $C = \{c_1, c_2, \dots, c_k\}$

Solution: classification function $\gamma : \mathbf{X} \rightarrow C$

Measure: misclassification

[PF] “classification predicts whether something will happen, whereas regr. predicts how much something will happen.”

SUPERVISED VERSION

Feature vector $\mathbf{x}$																	Target y					
$\mathbf{x}_1$	1	0	0	...	1	0	0	0	...	0.3	0.3	0.3	0	...	13	0	0	0	0	...	5	y_1
$\mathbf{x}_2$	1	0	0	...	0	1	0	0	...	0.3	0.3	0.3	0	...	14	1	0	0	0	...	3	y_2
$\mathbf{x}_3$	1	0	0	...	0	0	1	0	...	0.3	0.3	0.3	0	...	16	0	1	0	0	...	1	y_3
$\mathbf{x}_4$	0	1	0	...	0	0	1	0	...	0	0	0.5	0.5	...	5	0	0	0	0	...	4	y_4
$\mathbf{x}_5$	0	1	0	...	0	0	0	1	...	0	0	0.5	0.5	...	8	0	0	1	0	...	5	y_5
$\mathbf{x}_6$	0	0	1	...	1	0	0	0	...	0.5	0	0.5	0	...	9	0	0	0	0	...	1	y_6
$\mathbf{x}_7$	0	0	1	...	0	0	1	0	...	0.5	0	0.5	0	...	12	1	0	0	0	...	5	y_7
	A	B	C	...	TI	NH	SW	ST	...	TI	NH	SW	ST	...	Time	TI	NH	SW	ST	...		
	User				Movie					Other Movies rated						Last Movie rated						

Fig. 1. Example (from Rendle [2010]) for representing a recommender problem with real valued feature vectors $\mathbf{x}$. Every row represents a feature vector $\mathbf{x}_i$ with its corresponding target y_i . For easier interpretation, the features are grouped into indicators for the active user (blue), active item (red), other movies rated by the same user (orange), the time in months (green), and the last movie rated (brown).

Estimate the rating for the *new* user/film combination $\mathbf{x}_8$: most cells are 0, y_8 is unknown.

We face *sparsity*.

$$\mathbf{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots\}$$

Find rating estimate function $Y : \mathbb{R}^n \rightarrow T$ s.t.

- $T = \mathbb{R}$ for regression,
- $T = \{+, -\}$ for classification.

$$\hat{\mathbf{D}} = \{(\mathbf{x}^{(m+1)}, Y(y^{(m+1)}), (\mathbf{x}^{(m+2)}, Y(y^{(m+2)}), \dots\}$$

Note: Rendle uses different letters; here $n = \text{dimensions}(\mathbf{D})$

FOR REFERENCE: THE CONSTRAINTS SCENARIO

$$\mathbf{D} = \{\mathbf{x}^{(a)}, \mathbf{x}^{(b)} \dots\}$$

re-arrange the rows so that $\mathbf{x}^{(a)}$ *maps* higher than $\mathbf{x}^{(b)}$ and so on.

Ideal for Top-k searches and recommendations

THE MODEL

INTUITION

extend linear regression to capture *synergetic* effects between variables:

introduce a minimal quadratic effect $x_i x_j$

fill the table by looking at values on the same row or column of the target cell

GENERAL ESTIMATION

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^n w_i x_i$$

an initial (fixed) bias + linear regression.

To look at quadratic interactions, fix $d = 2$:

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n w_{ij} x_i x_j$$

- lots of training to find out all n^2 coefficients w_{ij}
- the w_{ij} 's may not even be significant (too close to 0)
- computing even a single prediction costs $\Theta(n^2)$

A SIMPLER MODEL

IN PRACTICE

1. fix $d=2$ and a small integer k (e.g., # of genres)
2. build a model of how the n dimensions relate to the k genres: a $V_{n \times k}$ matrix

$$W = V \cdot V^T \Rightarrow w_{ij} = \mathbf{v}_i^T \cdot \mathbf{v}_j = \langle \mathbf{v}_i, \mathbf{v}_j \rangle$$

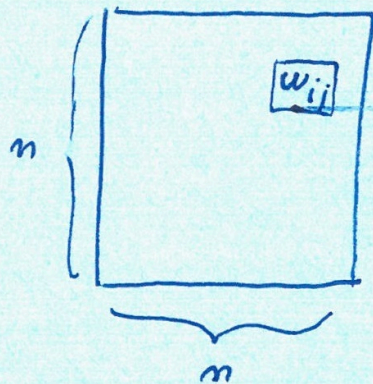
Key point: W contains $\frac{n^2}{2} - \frac{n}{2}$ estimates while the equivalent V only has $n \cdot k$ (latent) estimates.

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

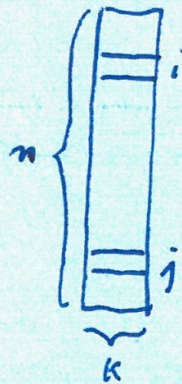
Where the inner/dot product is

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \mathbf{v}_i^T \cdot \mathbf{v}_j = \sum_{f=1}^k v_{if} v_{jf}$$

F-2-F:



F-2-L



$$w_{ij} x_i x_j \approx \bar{v}_i \cdot \bar{v}_j^T \cdot x_i \cdot x_j$$

[RENDLE, 2010]

$\hat{w}_{i,j} := \langle \mathbf{v}_i, \mathbf{v}_j \rangle$ models the interaction between the i -th and j -th variable.

Instead of using an own model parameter $w_{i,j} \in \mathbb{R}$ for each interaction, the FM models the interaction by factorizing it.

We will see later on, that this is the key point which allows high quality parameter estimates of higher-order interactions ($d \geq 2$) under sparsity.

COMPUTATIONAL COSTS

TH: COST IS LINEAR IN N

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

where

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \mathbf{v}_i^T \cdot \mathbf{v}_j = \sum_{f=1}^k v_{if} v_{jf}$$

How can this be computed in $\Theta(kn) = \Theta(n)$ iteration?

$$\sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

Insight: i and j never appear *together*: their iteration can be separated.

Idea: iterate over k outside, push i and j iterations inside.

IMPLEMENTATIONS

THE LIBFM SOURCE

libfm.org is the repository for the 'official' C++ implementation of FMs, which ended in 2014.

FMS IN PYTHON

PyFM

provides a new environment for running FMs within Python.

```
1 pip install git+https://github.com/coreylynch/pyFM
```

```
1 # Build and train a Factorization Machine
2 myfm = pylibfm.FM(num_factors=10,
3                   num_iter=100,
4                   task="regression",
5                   ...)
6
7 myfm.fit(X_train,y_train)
8 ...
```

1-HOT ENCODING

```
1 from pyfm import pylibfm
2 from sklearn.feature_extraction import DictVectorizer
3 import numpy as np
4
5 train = [
6     {"user": "1", "item": "5", "age": 19},
7     {"user": "2", "item": "43", "age": 33},
8     ...
9 ]
```

four users, four items: 8 columns

```
1 v = DictVectorizer()
2 X = v.fit_transform(train)
3 print(X.toarray())
4 [[ 19.  0.  0.  0.  1.  1.  0.  0.  0.]
5  [ 33.  0.  0.  1.  0.  0.  1.  0.  0.]
6  ...
7  ]
```

What is the estimated appreciation of user 1, aged 24 now, for item 10 once he or she buys it?

```
1 y = np.repeat(1.0, X.shape[0])
2
3 fm = pylibfm.FM()
4
5 fm.fit(X, y)
6
7 fm.predict(v.transform({"user": "1", "item": "10", "age": 24}))
```


CODA: THE TIME-EFFICIENCY OF MF

COMPUTATIONAL ASPECTS

Rendle proved that an intrinsically quadratic activity: compute all possible second-order, $x_i \cdot x_j$, effects, can be done in a time linear and not quadratic in n .

$$\hat{y}(\mathbf{x}) = w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n w_{ij} x_i x_j$$

To do so, Rendle models feature interactions by learning k latent factors:

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \sum_{f=1}^k v_{i,f} v_{j,f}$$

OPTIMISATION

While computing the mathematical formula for polynomial regression takes $\Theta(n^2)$ ops., Rendle does it in $\Theta(kn)$.

Notice how summing over different pairs is equivalent to summing over all pairs minus the self-interactions (divided by 2):

a correction factor $\frac{1}{2}$ is introduced from the beginning of the derivation.

$$\begin{aligned} & \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j - \frac{1}{2} \sum_{i=1}^n \langle \mathbf{v}_i, \mathbf{v}_i \rangle x_i x_i \end{aligned}$$

STEPS

$$\begin{aligned}& \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \\&= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j - \frac{1}{2} \sum_{i=1}^n \langle \mathbf{v}_i, \mathbf{v}_i \rangle x_i x_i \\&= \frac{1}{2} \left(\sum_{i=1}^n \sum_{j=1}^n \sum_{f=1}^k v_{i,f} v_{j,f} x_i x_j \right) - \frac{1}{2} \left(\sum_{i=1}^n \sum_{f=1}^k v_{i,f} v_{i,f} x_i x_i \right) \\&= \frac{1}{2} \left(\sum_{i=1}^n \sum_{j=1}^n \sum_{f=1}^k v_{i,f} v_{j,f} x_i x_j - \sum_{i=1}^n \sum_{f=1}^k v_{i,f} v_{i,f} x_i x_i \right)\end{aligned}$$

$$\begin{aligned}
&= \frac{1}{2} \left(\sum_{i=1}^n \sum_{j=1}^n \sum_{f=1}^k v_{i,f} v_{j,f} x_i x_j - \sum_{i=1}^n \sum_{f=1}^k v_{i,f} v_{i,f} x_i x_i \right) \\
&= \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right) \left(\sum_{j=1}^n v_{j,f} x_j \right) - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right) \\
&= \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right)^2 - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right)
\end{aligned}$$

Now the summations in i are inside the k summation but separated from each other.

Substituting back into the factorization machine formula:

$$\hat{y}(\mathbf{x}) = w_0 + \sum_{i=1}^n w_i x_i + \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right)^2 - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right)$$